Business Intelligence & Sales Analytics Platform

End-to-end project report: Amazon web scraping → MySQL → Power BI. This production-style pipeline sources raw product data directly from Amazon (live market signals) and transforms it into a decision-ready executive dashboard. Tools used: Python (Scrapy, Selenium, Pandas), SQLAlchemy, MySQL, Power BI Desktop. The project covers three phases: Data Collection, Data Storage & SQL Analysis, and Power BI Reporting.

Project Overview — Purpose & Scope

Objective

Design and deliver a complete BI pipeline that ingests live Amazon product data and produces an interactive executive Power BI dashboard reflecting pricing, demand signals, and category performance.

Scope

Three phases: (1) Data engineering (scraping & storage), (2) Data transformation (SQL analysis), (3) Business reporting (Power BI dashboards).

Why live scraping?

Using real Amazon data ensures analyses reflect actual market pricing, customer demand signals, category mix and dynamic discounts—not a static, pre-packaged dataset.

Phase 1 — Data Collection Architecture (Scrapy + Selenium)

Scrapy Spider (amazon_products)

Structured Scrapy project handles bulk scraping across 20 product categories (Technology, Furniture, Home Appliances, Apparel). Runs at low concurrency with randomized 3–9s delays to mimic human browsing and reduce blocking risk.

Selenium Fallback

Standalone Selenium script drives a real Chrome browser via webdriver-manager for JavaScript-rendered pages that require real browser rendering to capture content.

Anti-Block Middleware

Custom downloader middleware rotates realistic User-Agent profiles, detects CAPTCHA/blocked pages, saves debug HTML for selector troubleshooting, and isolates blocked responses for analysis.

New_Query_1781425329601.sql ● | generate_sales_data.py | selenium_scraper.py ●

selenium_scraper.py > ...

```python
import os
import re
import csv
import time
import random
import datetime
import logging
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException, NoSuchElementException
from webdriver_manager.chrome import ChromeDriverManager

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
logger = logging.getLogger(__name__)



KEYWORDS = ["laptop", "wireless earbuds", "office chair", "smart watch", "bluetooth speaker"]
PAGES_PER_KEYWORD = 2
HEADLESS = True
MIN_DELAY = 4
MAX_DELAY = 9
# ---------------------------------------------------------------------


USER_AGENTS = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/123.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:125.0) Gecko/20100101 Firefox/125.0",
]
```

(module) selenium

File Edit Selection View Go Run ··· Amazon_Sales_Analytics

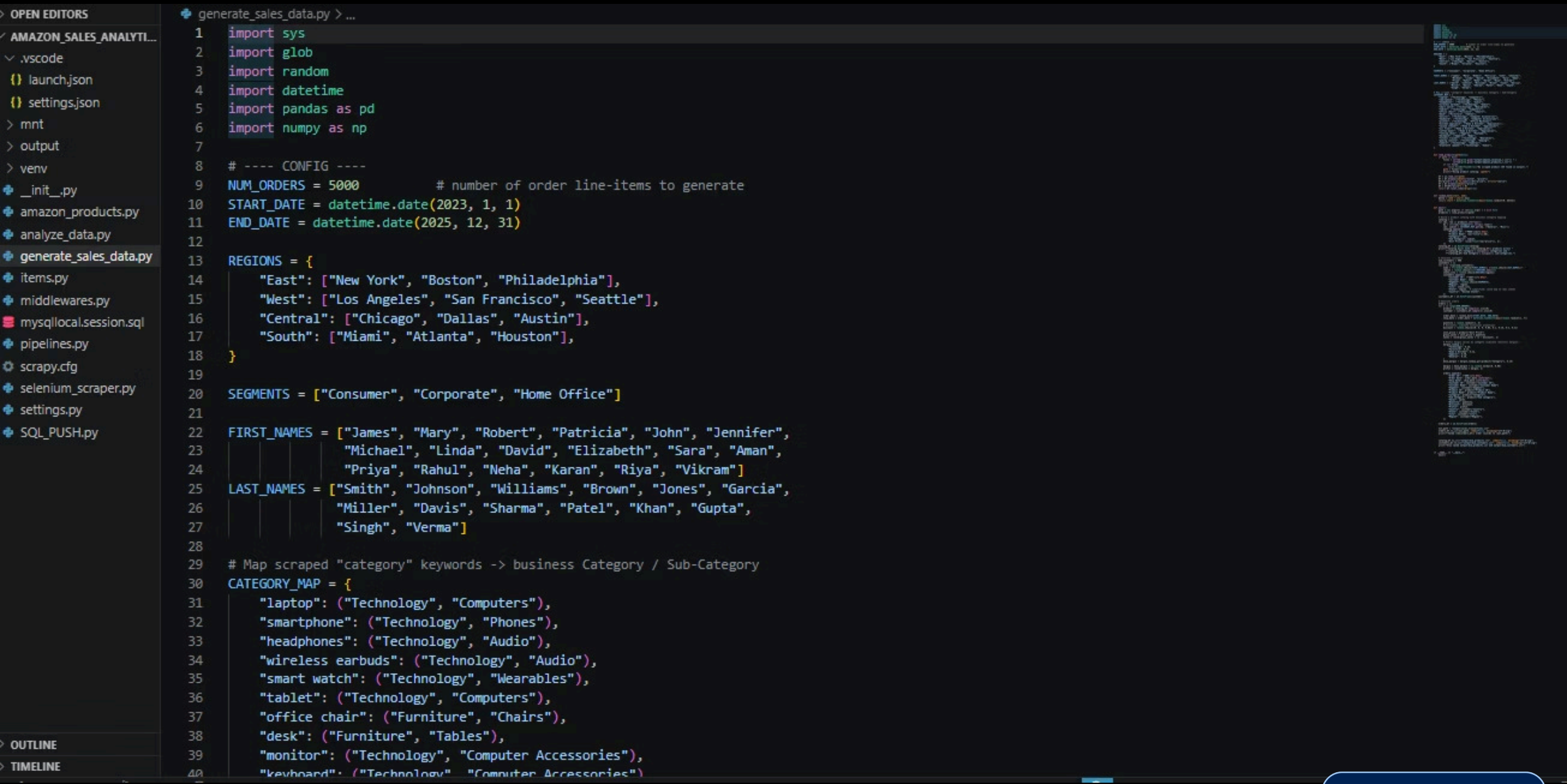
New_Query_1781425329601.sql ● | generate_sales_data.py | selenium_scraper.py ●

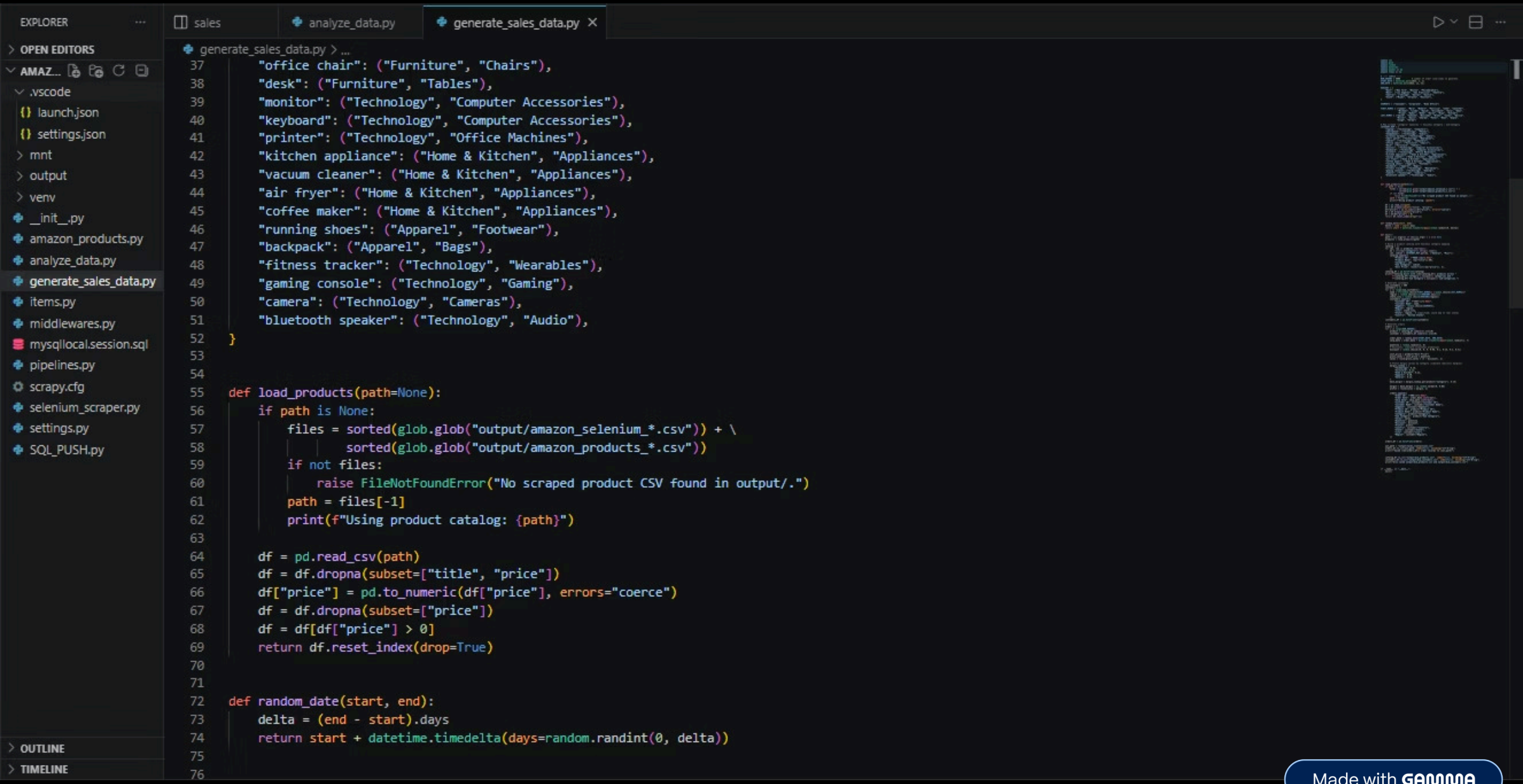
selenium_scraper.py > ...

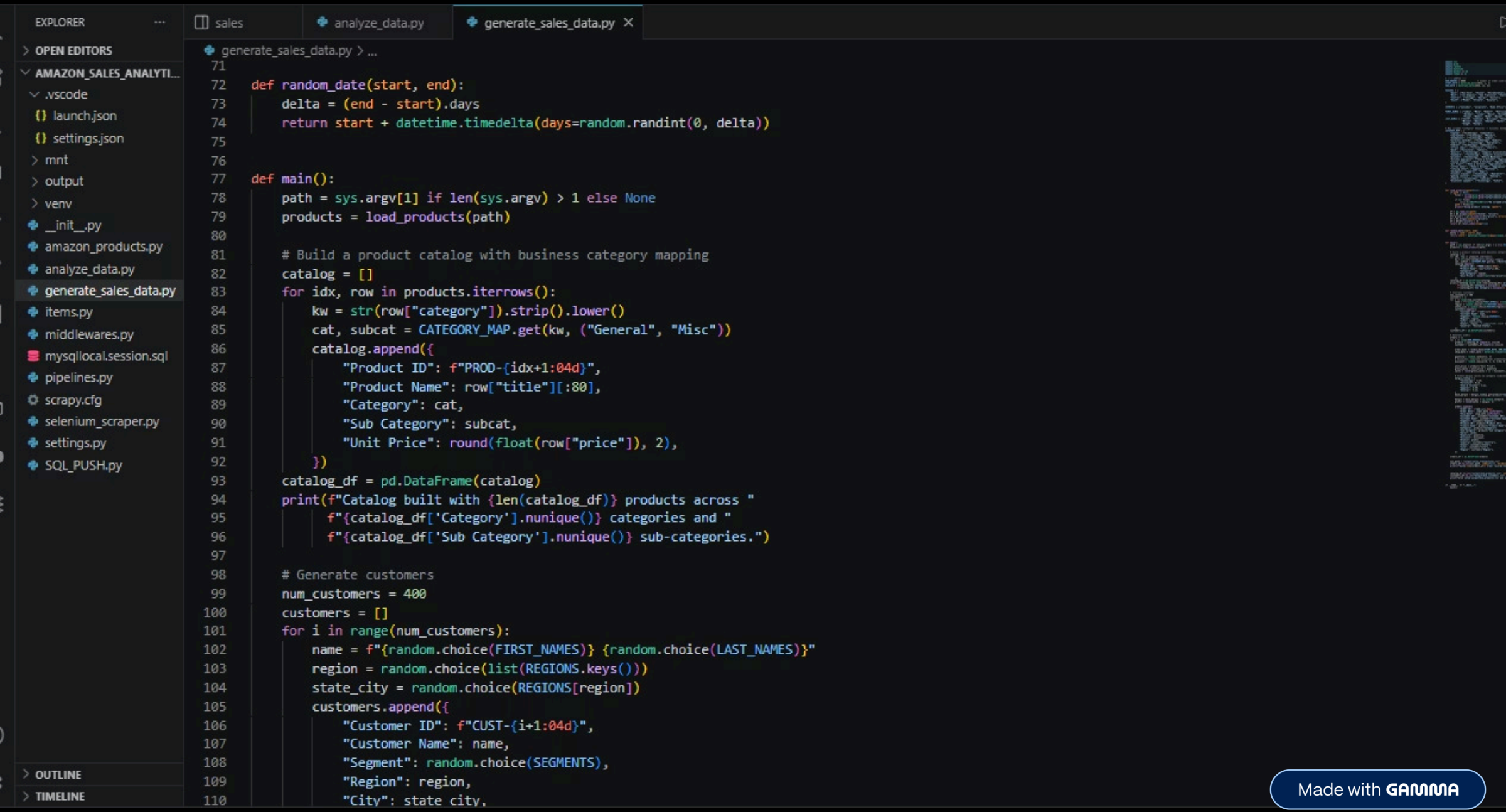
```python
def build_driver():···


def to_float(text):
    if not text:
        return None
    match = re.search(r"[\d,]+\.?\d*", text.replace(",", ""))
    return float(match.group()) if match else None


def to_int(text):
    if not text:
        return None
    text = text.upper().replace(",", "")
    multiplier = 1
    if "K" in text:
        multiplier = 1000
        text = text.replace("K", "")
    elif "M" in text:
        multiplier = 1_000_000
        text = text.replace("M", "")
    match = re.search(r"[\d.]+", text)
    return int(float(match.group()) * multiplier) if match else None


def parse_results_page(driver, keyword):···


def main():···


if __name__ == "__main__":
    main()
```

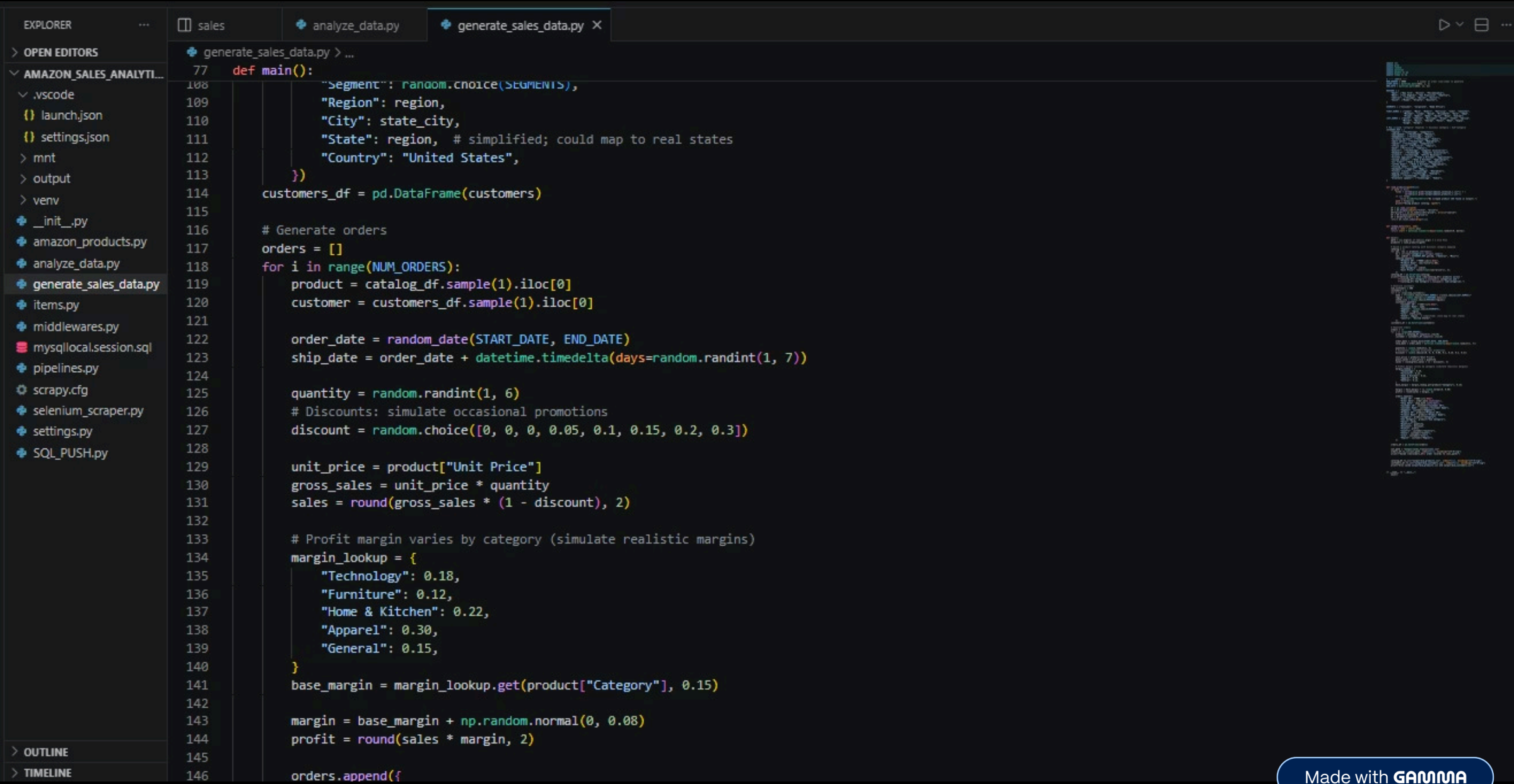
Phase 1 — Data Model & Export

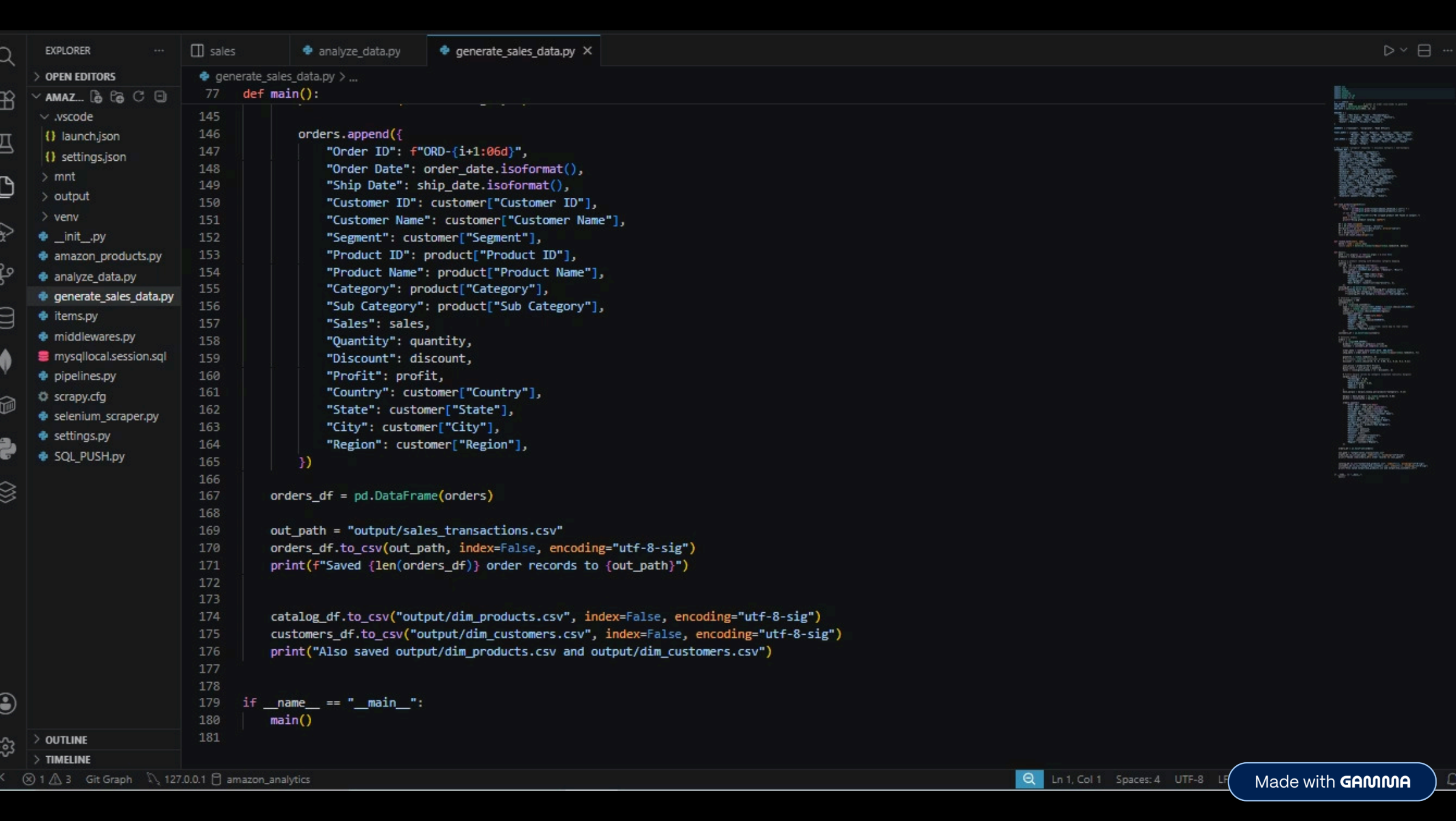
Each product record captures ASIN, title, brand, category, current price, original (MRP) price, discount %, average star rating, review count, 'bought last month' demand signal, Prime eligibility, and UTC timestamp. A custom Scrapy pipeline performs inline type conversion (prices → float, review counts parse K/M suffixes) and exports timestamped UTF-8 CSV files.










EXPLORER
OPEN EDITORS
AMAZ...
.vscode
launch.json
settings.json
mnt
output
venv
__init__.py
amazon_products.py
analyze_data.py
generate_sales_data.py
items.py
middlewares.py
mysqllocal.session.sql
pipelines.py
scrapy.cfg
selenium_scraper.py
settings.py
SQL_PUSH.py
OUTLINE
TIMELINE
sales
analyze_data.py
generate_sales_data.py
generate_sales_data.py > ...
77 def main():
145
146 orders.append({
147 "Order ID": f"ORD-{i+1:06d}",
148 "Order Date": order_date.isoformat(),
149 "Ship Date": ship_date.isoformat(),
150 "Customer ID": customer["Customer ID"],
151 "Customer Name": customer["Customer Name"],
152 "Segment": customer["Segment"],
153 "Product ID": product["Product ID"],
154 "Product Name": product["Product Name"],
155 "Category": product["Category"],
156 "Sub Category": product["Sub Category"],
157 "Sales": sales,
158 "Quantity": quantity,
159 "Discount": discount,
160 "Profit": profit,
161 "Country": customer["Country"],
162 "State": customer["State"],
163 "City": customer["City"],
164 "Region": customer["Region"],
165 })
166
167 orders_df = pd.DataFrame(orders)
168
169 out_path = "output/sales_transactions.csv"
170 orders_df.to_csv(out_path, index=False, encoding="utf-8-sig")
171 print(f"Saved {len(orders_df)} order records to {out_path}")
172
173
174 catalog_df.to_csv("output/dim_products.csv", index=False, encoding="utf-8-sig")
175 customers_df.to_csv("output/dim_customers.csv", index=False, encoding="utf-8-sig")
176 print("Also saved output/dim_products.csv and output/dim_customers.csv")
177
178
179 if __name__ == "__main__":
180 main()
181
1 3 Git Graph 127.0.0.1 amazon_analytics
Ln 1, Col 1 Spaces: 4 UTF-8
Made with GAMMA

Phase 2 — Storage & Synthetic Sales Generation (MySQL + SQLAlchemy)

Cleaned product CSVs were loaded into MySQL (amazon_analytics) using pandas + SQLAlchemy to avoid encoding issues. A supplementary Python script (generate_sales_data.py) used the scraped product catalog to generate a realistic 5,000-row transactional sales dataset with Order Date, Customer ID, Segment, Region, Sales, Discount, and Profit—creating a proper analytics-ready fact table for SQL and Power BI.

Phase 2 — SQL Techniques & Key Queries

SQL techniques used to extract business insights:

Window Functions

RANK() OVER (PARTITION BY category) for top-5 revenue products; SUM() OVER for running totals; LAG() for month-over-month growth.

CTEs & Multi-step Logic

CTEs for customer segmentation cohorts, ABC inventory classification (A = top 80% revenue), and multi-stage analyses.

Profitability Analysis

Discount-band grouping quantified correlation between deep discounts (>20%) and negative profit margins; results fed into dashboard alerts.

Example: Month-over-month revenue growth using LAG() (query applied to the generated sales fact table to compute pct_growth by month).

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- amazon_analytics
  - Tables
    - dim_customers
    - dim_products
    - products
    - sales
      - Columns
        - Order ID
        - Order Date
        - Ship Date
        - Customer ID
        - Customer Name
        - Segment

Administration Schemas

Information

No object selected

dim_customers SQL File 10* SQL File 11*

Don't Limit

```sql
-- Discount impact analysis — does higher discount actually hurt profit?
SELECT
    CASE
        WHEN Discount = 0 THEN 'No Discount'
        WHEN Discount <= 0.1 THEN 'Low (≤10%)'
        WHEN Discount <= 0.2 THEN 'Medium (≤20%)'
        ELSE 'High (>20%)'
    END AS discount_band,
    COUNT(*) AS num_orders,
    AVG(Profit) AS avg_profit,
    AVG(Sales) AS avg_sales,
    ROUND(AVG(Profit)/AVG(Sales)*100,2) AS avg_margin_pct
FROM sales
GROUP BY discount_band
ORDER BY avg_margin_pct DESC;
```

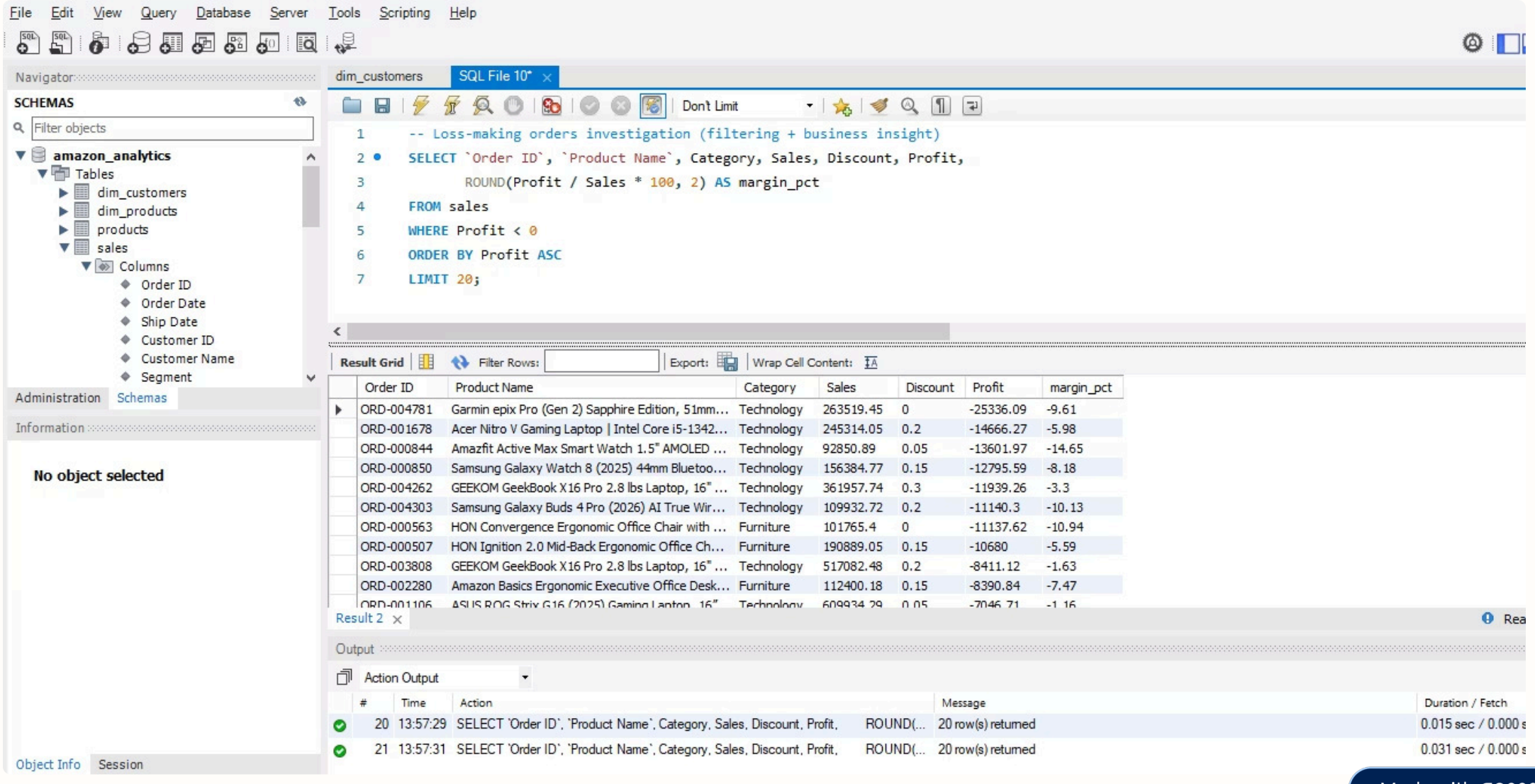
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

discount_band	num_orders	avg_profit	avg_sales	avg_margin_pct
Low (≤10%)	1273	14850.796535742345	89056.18641005507	16.68
High (>20%)	635	11273.983511811026	67972.29333858269	16.59
No Discount	1814	16204.022337375944	98499.13377067221	16.45
Medium (≤20%)	1278	13248.618528951512	81757.6169170578	16.2

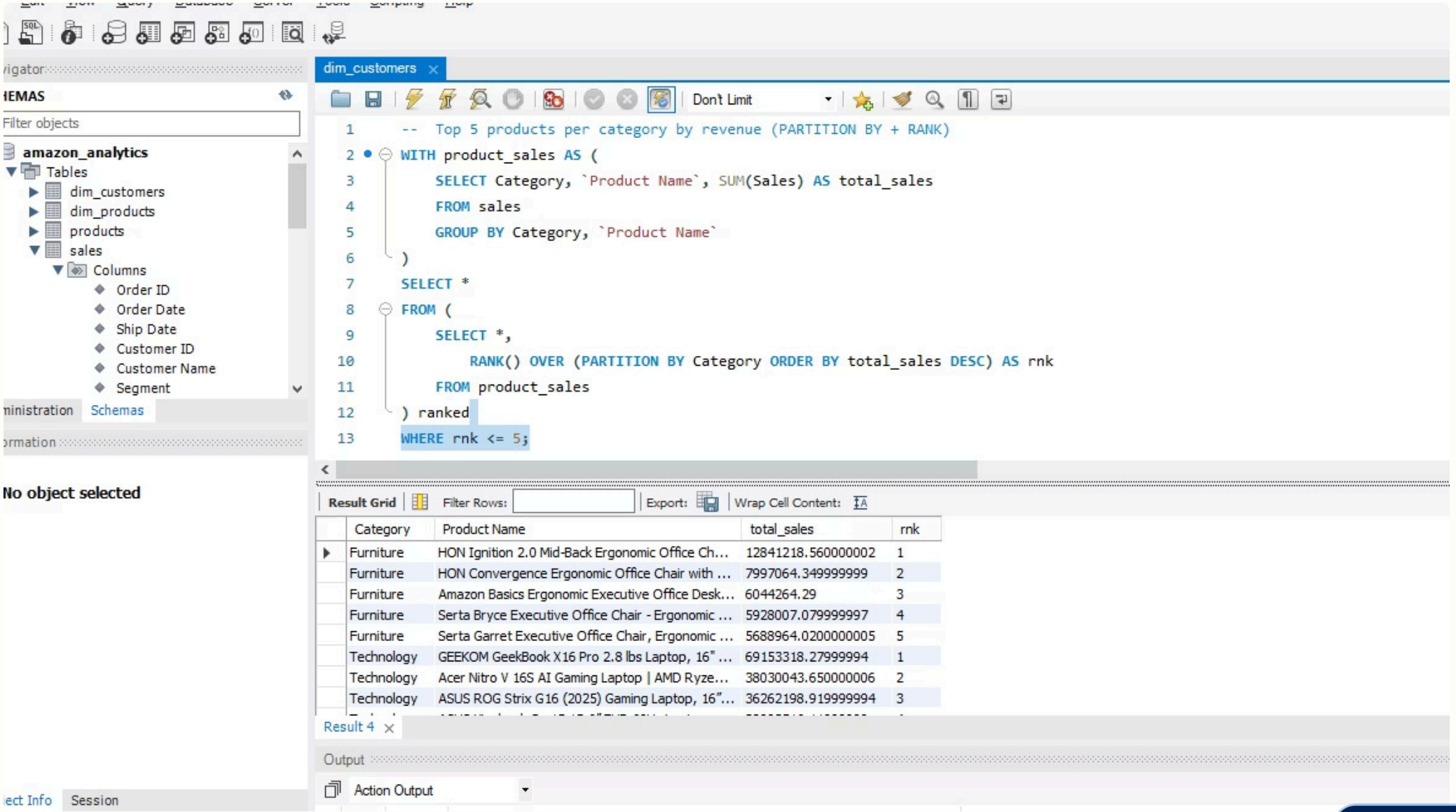
Result 2

Read Only

SQL Query 2

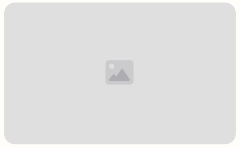


SQL Query 3



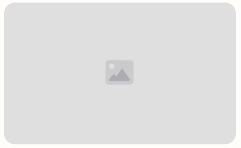
Phase 3 — Power BI Report & Modelling

The cleaned and transformed dataset was loaded into Power BI Desktop via Text/CSV. A DateTable was created in DAX and marked as a Date Table. Relationships were defined between sales fact and dimension tables (products, customers) to implement a star schema. The final report includes two pages: Executive Summary and Profitability Deep-Dive.



Executive Summary (Page)

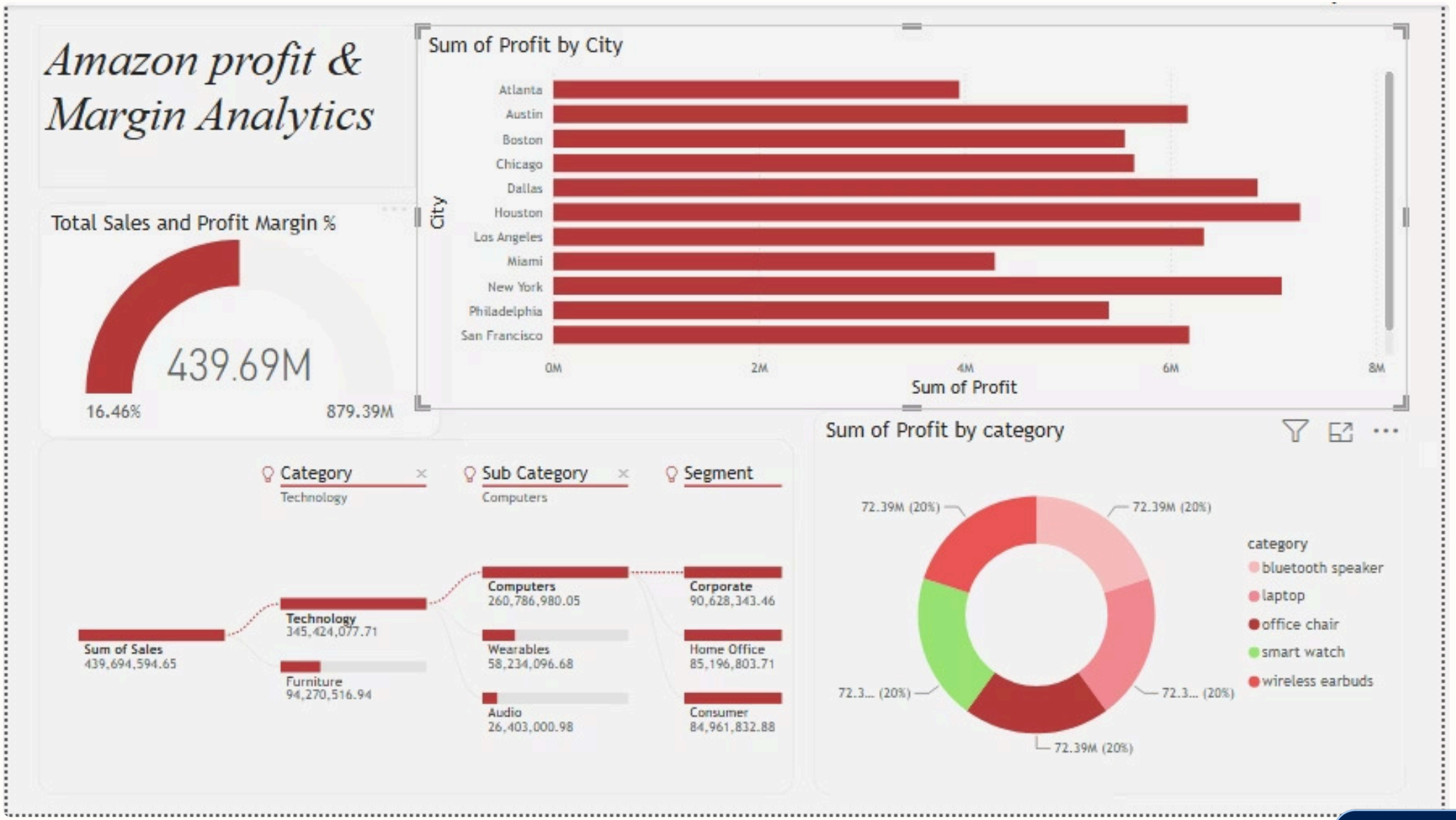
KPI cards: Total Sales 439.69M, Profit 72.39M, Margin 16.46%, Orders 5K; area chart Sales vs Profit; USA map by city; segment donut chart. Key headline: Sales YoY growth 49.64%.



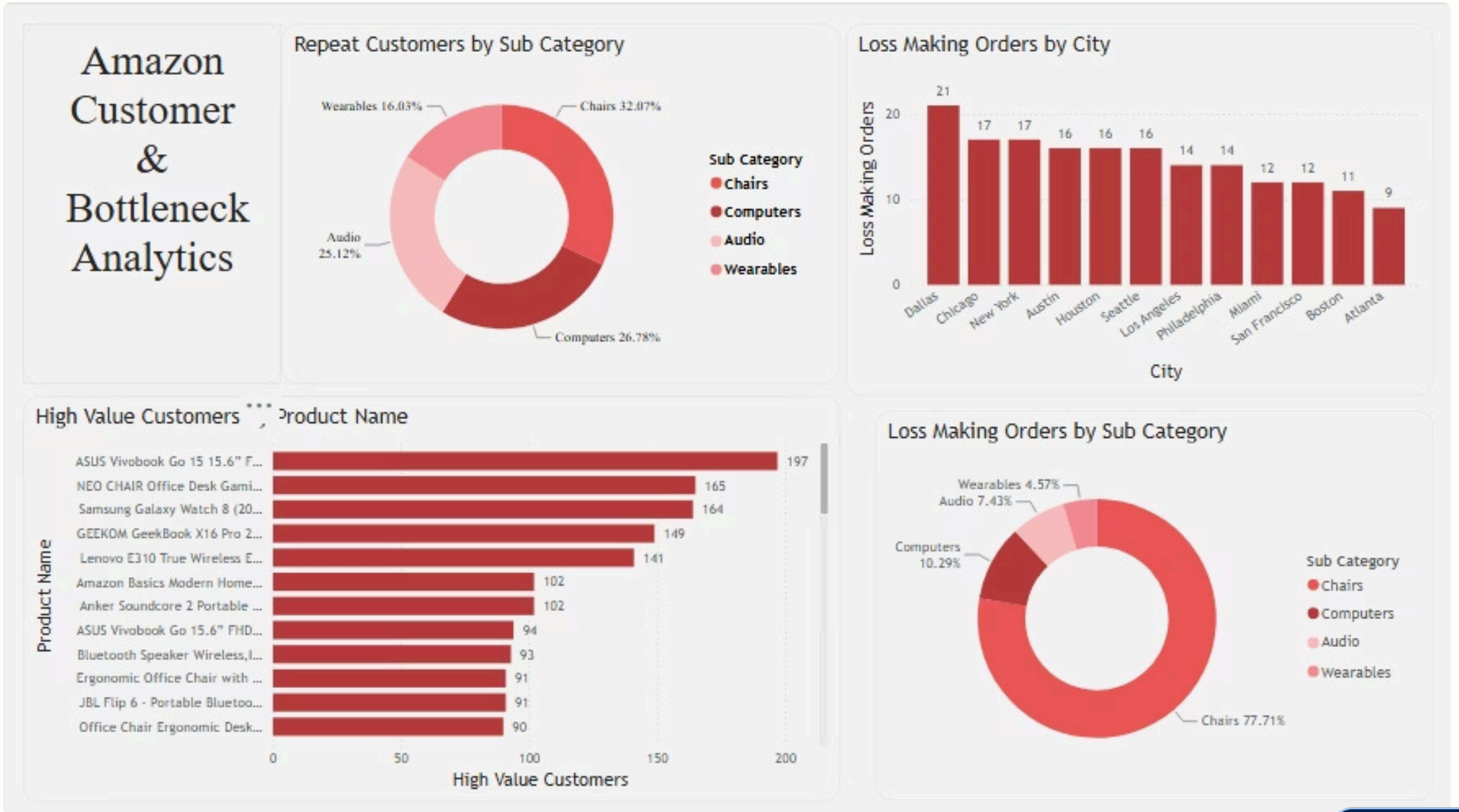
Profitability Deep-Dive (Page)

Profit by city bar chart, profit by category donut, Decomposition Tree: Total Sales → Category → Sub-Category. Findings show Technology dominance and city-level concentration risks.

Power BI Dashboard 3: Profit & Margin Analytics

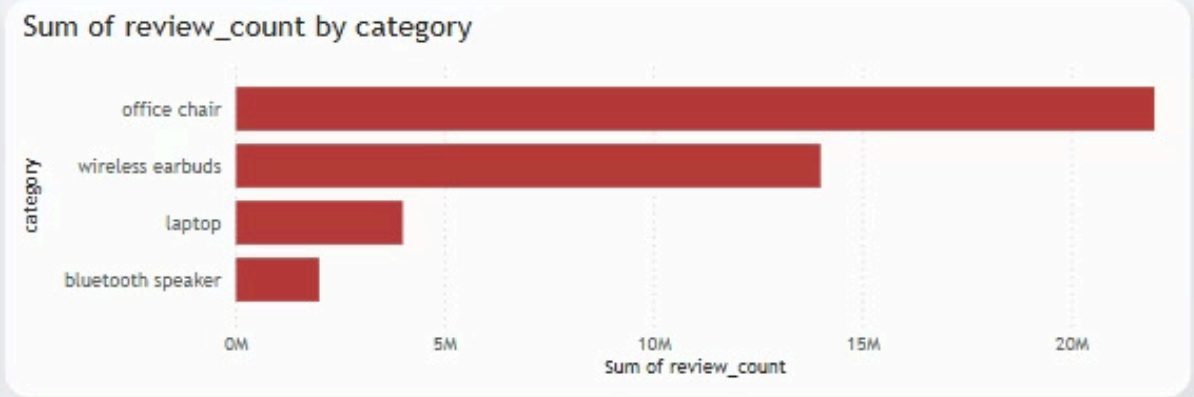


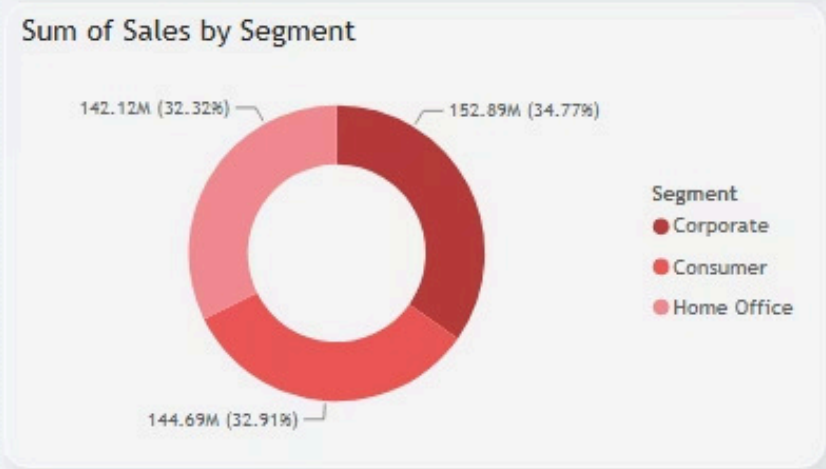
Power BI Dashboard 2: Customer & Bottleneck Analytics

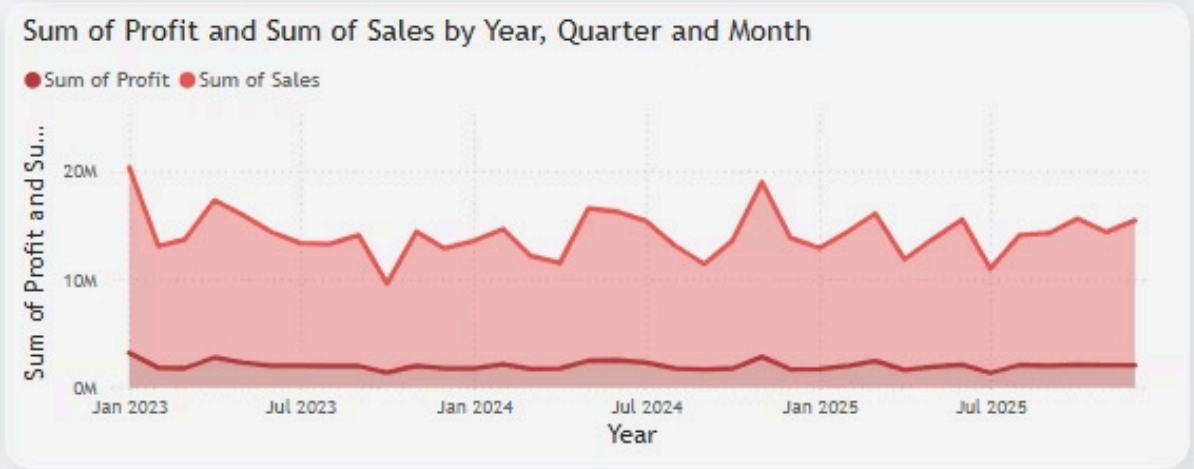


Power BI Dashboard 1: Executive Summary

87.94K	294M	439.69M	16.46%	49.64%	5K	72.39M
Avg Order Value	growthSales LY	Total Sales	Profit Margin %	Sales YoY %	Total Orders	Total Profit







DAX Measures & Analytical Logic

Core DAX measures developed to power dynamic KPIs and slicers:

- Profit Margin %: DIVIDE([Total Profit], [Total Sales]) — surfaces margin compression even when raw profit looks healthy.
- Sales YoY %: DIVIDE([Total Sales] - CALCULATE([Total Sales], SAMEPERIODLASTYEAR(DateTable[Date])), [Sales LY]) — requires proper DateTable.
- Customer Tier: SWITCH(TRUE(), ...) — classifies customers into High/Medium/Low based on spend relative to average for segmentation analysis.
- Loss Making Orders: CALCULATE(COUNTROWS(sales), sales[Profit] < 0) — quantifies discount-driven margin erosion immediately.

Key Business Findings

Revenue Concentration

Technology dominates revenue (345M, ~79% of total). Within Technology, Computers account for 260M (major share), creating category concentration risk.

Profit vs. Discount

Discounts >20% consistently correlate with negative profit margins across categories — confirmed by both SQL and DAX analyses.

Geographic Risk

Seattle, Austin, and New York are top three profit-generating cities; regional disruption could disproportionately impact performance.

Segment Dynamics

Consumer (34.77%), Home Office (32.91%), Corporate (32.32%) are near-equal splits by revenue, but Corporate orders have higher AOV — a lever for growth prioritization.

Demand Signals

'Bought last month' counts for high-rated (4.5+) products in Wearables and Audio categories far exceed Furniture — indicating stronger organic demand momentum in tech accessories vs big-ticket items.

Skills Demonstrated & Final Thoughts

1 Web Scraping

Scrapy project with custom middlewares, Selenium fallback, anti-bot mitigations, incremental UTF-8 CSV export pipeline.

2 Python / Pandas

Data cleaning, type normalization, EDA, SQLAlchemy ETL, synthetic sales generation to create realistic business distributions for analytics.

3 SQL

CTEs, window functions (RANK, LAG, running totals), cohort and segmentation logic, ABC analysis, discount-impact queries used to surface actionable insights.

4 Power BI

Star-schema modelling, DateTable, DAX measures (time-intelligence, SWITCH segmentation, DIVIDE), KPI cards, Decomposition Tree, geographic mapping for executive reporting.

5 Business Impact

Identified discount-margin correlation, geographic concentration risk, segment AOV differences, and category-level demand signals — enabling prioritized strategic actions.

This end-to-end project demonstrates an operational BI pipeline from raw data sourcing to decision-ready visual reporting. Next steps: refine anti-blocking strategies for long-term data collection, implement alerting for discount-driven losses, and prioritize growth initiatives targeting Corporate accounts and high-margin product lines.